

Requirments document

Author: Miroslav Denkov

Table of Contents

1. Introduction	1
2. Assignment Description	1
3. Scope	1
4. Requirements	2

Date	Version	Changes	Status
18/03/2026	1	• Created document	Draft

Table 1: Change log

1. Introduction

This document serves the purpose of providing information in the requirements of the project, to be developed by Miroslav Denkov(author of this document).

2. Assignment Description

The assignment consists in the continuation of a self-developed API protocol for controlling the Ufactory Lite 6 robot arm. The arm should be able to execute its tasks without making collisions with the other equipment in the lab. The arm must be able to walk through its trajectory unimpeded. The demonstration includes the arm calibrateing itself and pick up the small plastic boxes, where the solutions would be placed. The Technologies involved are trajectory planning, inverse kinematics,Intermachine Communication, :::ROS2 framework and 🐍 Python.

3. Scope

The Project includes	The project does not include
Research on :::ROS2 and similar technology	Explicit development on SILA2
Decision and consultation with backed-up evicence on which techonolgy is better-suited for the task at hand.	Works on connecting the other equipment
Implementation of path trajectory algorithm using either default tooling or :::ROS2	
Demonstration of the robotarm's capabilities to safely deliver payload from point A to B.	

Table 2: Project Scope

4. Requirements

Functional Requirements	Non-functional Requirements
- The arm must be able to position itself(calibration). - The arm must NOT collide with surrounding equipment or itself(over-reaching). - The arm must be able to find its trajectory. - The arm must pick the small container up and put it on its destination.	- A conclusion towards tooling. Should the default tooling from Ufactory or ROS framework be used? - Conclude if it is beneficial to use ROS2's MoveIT library for trajectory planning(Part of ROS framework implementation experiments). - Conclude what calibration implementation would be the most beneficial. QR-target-based or something else. - Version control with Git. - Research topics with DOT.

Table 3: Functional and Non-functional requirements

Must	Should	Could	Won't
The system must plan its trajectory	The system should have automatic positioning calibration		
The system must plan its trajectory			
The system must be able to detect joint movement limit			
The system must be able to detect joint movement limit			The system should have automatic positioning calibration

Table 4: MoSCoW table